Motivation

- Un algorithme a pour objectif la résolution d'un problème.
- Est-ce que l'algorithme donne...
  1. une réponse ? **terminaison**
  2. la bonne réponse ? **correction**
  3. la bonne réponse en un temps acceptable ? **complexité**

3

Motivation

- On veut qu'un algorithme soit **correct**, mais aussi **efficace**, c'est-à-dire :
  - ✓**rapide** (en termes de temps d'exécution) ;
  - ✓**économe** en ressources (espace de stockage, mémoire utilisée) ;
- On a donc besoin d'outils qui nous permettront d'évaluer la qualité **théorique** des algorithmes proposés ;
- On se focalisera sur le temps d'exécution ;

4

Exemple : deux méthodes de recherche dans les listes

- Voici deux méthodes vérifiant si un élément appartient à une liste triée :

Version courte

```
def f(T, x):
    if x in T:
        return True
    return False
```

Version longue

```
def g(T, x, a=0, b=None):
    if b is None:
        b = len(T)-1

    if a > b:
        return False

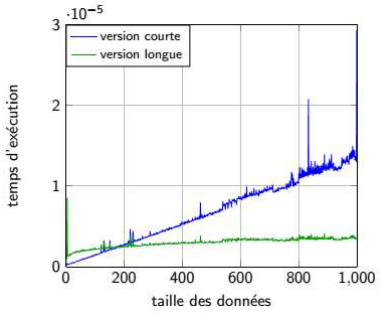
    p = (a + b) // 2
    if T[p] == x:
        return True
    if T[p] > x:
        return g(T, x, a, p-1)
    return g(T, x, p + 1, b)
```

Laquelle choisir ?

5

Exemple : deux méthodes de recherche dans les listes

- La mesure du temps d'exécution en pratique nous aide :



... mais ne nous dit pas pourquoi l'une est meilleure que l'autre.

6

Théorie de la complexité

- La **théorie de la complexité** (algorithmique) vise à répondre à ces besoins;
- Elle permet :
  1. de classer les problèmes selon leur **difficulté** ;
  2. de classer les algorithmes selon leur **efficacité**
  3. de **comparer** les algorithmes résolvant un problème donné afin de faire un choix éclairé sans devoir les implémenter ;

7

Evaluation de la rapidité d'un algorithme

- On ne mesure pas la durée en heures, minutes, secondes, ... :
  1. cela impliquerait d'implémenter les algorithmes qu'on veut comparer ;
  2. de plus, ces mesures ne seraient pas pertinentes car le même algorithme sera plus rapide sur une machine plus puissante ;
- Au lieu de ça, on utilise des **unités de temps abstraites** proportionnelles au nombre d'opérations effectuées ;
- Au besoin, on pourra ensuite adapter ces quantités en fonction de la machine sur laquelle l'algorithme s'exécute ;

8

Calcul du temps d'exécution

- Chaque **instruction basique** consomme une unite de temps ; (affectation d'une variable, comparaison, +, -, *, /, ...)
- Chaque **itération** d'une boucle rajoute le nombre d'unités de temps consommées dans le corps de cette boucle ;
- Chaque **appel de fonction** rajoute le nombre d'unités de temps consommées dans cette fonction ;
- Pour avoir le nombre d'opérations effectuées par l'algorithme, on additionne le tout ;

9

Exemple : calcul de la factorielle de n ϵ N

- Le programme suivant calcule n! = n*(n-1)*(n-2)*…*2*1 (avec 0! = 1) :

Exemple (factorielle de n)

```
def factorielle(n):
    fact = 1                          initialisation : 1
    i = 2                             initialisation : 1
    while i <= n:                     itérations : au plus n − 1
        fact = fact * i               multiplication + affectation : 2
        i = i + 1                     addition + affectation : 2
    return fact                       renvoi d'une valeur : 1
```

- On a aussi un test à chaque itération ;
- Nombre total d'opérations :

1+1+(n-1)*5+1=5n-2

10